The $10K Website Kit: How Claude Builds an Agency-Quality Site for Your Business

By the end of this guide, Claude will design, build, test and polish a full website for your business, and you'll know exactly how to steer it like a creative director.

Want more AI workflows like this?
Join our free community for the best AI tools, prompts, and shortcuts.
Join here: skool.com/viral-ads

What this actually is (in plain English)

Out of the box, Claude is like a brilliant junior developer with zero design taste. It writes working code fast, but every site it makes looks the same: a big centered banner at the top, a purple gradient, three cards, the same font.

Visitors have seen that template a thousand times, so they trust it less. This setup fixes it by giving Claude **a designer's rulebook, a motion expert's rulebook, and a pair of eyes**.

TWO WORDS YOU'LL SEE A LOT

> A skill is a rulebook (a folder of instructions) that Claude reads before it starts a task, the same way a new hire reads the brand guidelines. MCP stands for Model Context Protocol. Think of it as a power outlet: plug a tool into it and Claude can suddenly use that tool.

Here are the five pieces and the job each one does.

- **Claude Code.** The version of Claude that lives on your computer and can create files, run commands and build real projects. Think of it as Claude with hands.
- **Taste Skill** (by Leonxlnx). Picks the design direction before any code is written and bans the generic AI look. It's the art director.
- **Impeccable** (by Paul Bakaus). One skill with 24 commands like critique, layout, typeset and polish. It's the senior designer who reviews and fixes every page.
- **Emil Kowalski's skills.** A pack of 14 skills from a design engineer who worked at Vercel and Linear. They make animations feel smooth and expensive instead of cheap. He's the motion specialist.
- **Figma MCP + Playwright MCP.** Figma lets Claude read and write real design files. Playwright lets Claude open your site in a real browser, click every button and take screenshots. That's the QA tester.

What you end up with: you describe your business in a paragraph. Claude designs the pages, writes the copy and code, adds tasteful animation, opens the site in a browser, finds what's broken and fixes it.

Then you drag one folder onto a free host, and your site is live the same day, with a contact form that actually

sends you leads.

The thing nobody tells you

Most people think AI websites look cheap because the AI isn't smart enough. Wrong.

Claude can write almost any design you can describe. The problem is **defaults**. When you don't tell it exactly what good looks like, it falls back to the average of every template it has ever seen.

THE AHA

> Skills don't make Claude smarter. They delete its bad defaults and replace them with the specific rules a top designer would follow. That's why the same Claude, with the same prompt, suddenly produces a site that looks like an agency built it.

The second secret is **the loop**. Agencies never ship the first draft: they design, build, review, fix and test.

This setup gives Claude every step of that loop. Playwright closes it, because Claude can finally see its own work instead of guessing.

The order that works: Taste sets the direction, Claude builds, Impeccable critiques and polishes, Emil's skills fix the motion, Playwright tests everything on desktop and phone.

Skip a step and you get a site that looks great in one screenshot but breaks the moment a real visitor touches it.

Before you start: the 10-minute checklist

You need five things. Tick them off before you install anything, and the rest of this guide goes smoothly.

1. **A paid Claude plan (Pro or higher).**
   Claude Code needs a Pro, Max, Team or Enterprise account. The free plan doesn't include it. Sign up at claude.ai if you haven't already.
2. **Node.js.**
   A free program that lets your computer run the installer commands in this guide. Go to nodejs.org, click the button labeled LTS (the stable version), and click through the installer. Then close and reopen your terminal so it notices.
3. **Google Chrome.**
   Playwright opens your site in Chrome by default, so have it installed.
4. **A terminal.**
   The text window where you type commands. On Mac, open the Terminal app. On Windows, open PowerShell from the Start menu. You'll copy and paste every command from this guide, nothing from

memory.

5. **A Figma account (optional).**
   Only needed if you want Claude to work with design files. More on the limits below.

WINDOWS EXTRA (OPTIONAL)

> Install Git for Windows from git-scm.com/downloads/win. Claude Code works without it, but runs smoother with it, and it lets Claude save undo checkpoints for you.

THE "FREE" FINE PRINT

> All three design skills are free and open source. What costs money: a paid Claude plan to run Claude Code, and a paid Figma seat if you want heavy Figma use. Hosting can be free (see the launch step). So it's not literally free, but it's a tiny fraction of an agency invoice.

Step 1: Install Claude Code

Open your terminal and paste the one line for your computer. It downloads Claude Code and sets it up.

Mac or Linux

```
curl -fsSL https://claude.ai/install.sh | bash
```

Windows (PowerShell)

```
irm https://claude.ai/install.ps1 | iex
```

Close the terminal, open a fresh one, and check it worked. You should see a version number.

```
claude --version
```

Now make a folder for your website, step into it, and start Claude. The first time, a browser window opens so you can log in with your Claude account. If Claude asks whether you trust the files in this folder, choose Yes.

```
mkdir my-website
cd my-website

claude
```

Where that folder lives: inside your user folder. On Windows that's C:\Users\YOUR-NAME\my-website. On Mac, open Finder, click Go, then Home, and you'll see my-website. Everything Claude builds lands in there.

TWO PLACES YOU TYPE (READ THIS TWICE)

> The plain terminal is the window with a blinking line ending in $ or >. Install commands go there: npx, claude mcp add, claude plugin install.

THE SECOND PLACE: INSIDE CLAUDE

> Inside Claude is the chat box you see after you type claude. Slash commands like /mcp and /impeccable, and every plain-English prompt, go there. Type /exit to get back to the plain terminal.

Every time you come back to your project, on any day, open a terminal and run these two lines (in the plain terminal):

```
cd my-website

claude --continue
```

NOTHING GETS LOST

> Closing Claude never loses your site, because every file stays in the folder. Only the chat memory resets, and claude --continue picks the last conversation back up.

Step 2: Install the three design skills

Type **/exit** to leave Claude. Everything in this step runs in the **plain terminal**, inside your my-website folder.

Most of these use **npx skills**, the official skill installer made by Vercel. If it asks "Ok to proceed?", type y and press Enter. The -g part installs each skill for every project, so you only do this once.

GITHUB, STARS AND NPM IN 20 SECONDS

GitHub is where developers publish their projects. Each project is called a repo, and stars are its likes, so a high count is a trust signal. npm is the app store those installers come from, and npx is the command that grabs one and runs it. It came free with Node.js.

WINDOWS: IF POWERSHELL SAYS "RUNNING SCRIPTS IS DISABLED"

Windows blocks npx by default on some computers. The easy fix: type npx.cmd instead of npx at the start of the command, for example npx.cmd skills add ... and everything else stays the same.

A. Taste Skill (the art director)

```
npx skills add https://github.com/Leonxlnx/taste-skill --skill "design-taste-frontend" -g -a claude-code
```

Before writing code, Claude reads your brief, decides on a design language and sets three dials: how experimental the layout is, how much motion to use, and how packed each screen is. It also bans the classic AI tells.

TYPO-SQUAT ALERT

There's a package on npm called taste-skill that belongs to a different person, not the real Taste Skill. Always install with the full GitHub address exactly as shown above. Never type npx taste-skill.

B. Impeccable (the senior designer)

```
npx impeccable install
```

It shows the AI tools it found on your computer. Pick Claude, then choose to install globally (for all projects).

Prefer installing from inside Claude Code instead? Use these two lines inside Claude and pick Impeccable from the list. You only need one of the two methods.

```
/plugin marketplace add pbakaus/impeccable

/plugin
```

C. Emil Kowalski's motion skills (the animation specialist)

```
npx skills add emilkowalski/skills --skill emil-design-eng --skill
animate --skill review-animations --skill improve-animations --skill
break-ui -g -a claude-code
```

This installs the five that matter for websites: the main design rulebook, an animation builder, a strict animation reviewer, a site-wide animation audit, and break-ui, which stress-tests your pages with worst-case content.

Check it worked

Start Claude (type **claude** in the plain terminal) and run this once, inside Claude. Impeccable asks a few questions about your business and saves the answers in a file called PRODUCT.md, so every later command knows who the site is for.

```
/impeccable init
```

Then paste this inside Claude:

```
List every skill you have installed and one line on what each one does.
```

You should see design-taste-frontend, impeccable, emil-design-eng, animate, review-animations, improve-animations and break-ui. If one is missing, type /exit, rerun its command, and start Claude again.

Bonus: use two of the skills in the normal Claude app too

Taste and Emil's skills are plain instruction folders, so they also work at claude.ai. Impeccable runs a small engine on your computer, so keep that one in Claude Code.

1. **Download the skill.**
   On the GitHub page (links at the end), click the green Code button, then Download ZIP, and unzip it.
2. **Zip just one skill folder.**
   Open the skills folder inside and zip a single skill, for example taste-skill or emil-design-eng.
3. **Turn on code execution.**

In Claude, go to Settings, Capabilities, and switch on Code execution and file creation.

4. **Upload it.**
   Go to Customize, Skills, click +, then Create skill, then Upload a skill, and pick your ZIP.

Step 3: Connect Claude to Figma (the official way)

Figma runs its own official MCP server, hosted by Figma itself at mcp.figma.com. That's the only one you should use.

Type **/exit** if you're inside Claude, then paste this in the plain terminal. It installs Figma's plugin from Anthropic's official marketplace: the connection plus Figma's own skills.

```
claude plugin install figma@claude-plugins-official
```

Want the bare connection without Figma's extra skills? Use Figma's manual command instead (not both). The --scope user part makes it work in every project.

```
claude mcp add --scope user --transport http figma
https://mcp.figma.com/mcp
```

1. **Start Claude and type /mcp**
   You'll see a list of connected tools.
2. **Select figma, then Authenticate**
   A Figma page opens in your browser.
3. **Click Allow Access**
   Back in Claude you should see: Authentication successful. Connected to figma.

What it unlocks: paste a link to any Figma frame and Claude turns it into matching code. It can also write straight into Figma (frames, components, auto layout).

And it can capture your finished live site back into Figma as editable layers, so a designer or client can tweak it.

Two Figma words first: a tool call is one action Claude takes in Figma, like reading one frame. A seat is the type of Figma license you pay for.

THE FIGMA LIMIT NOBODY MENTIONS

> On the free Starter plan, or with a View or Collab seat, you get only 6 tool calls per month,

which is gone in one session. Real use needs a Dev or Full seat on a paid Figma plan. Writing into Figma is free during Figma's beta but will become a paid feature later.

YOU DON'T NEED FIGMA TO BUILD THE SITE

Figma is for two situations: you already have a design you want built pixel-perfect, or you want an editable design file to hand to a designer or client. If neither applies, skip this step. Claude plus the skills plus Playwright builds the whole site without it.

Step 4: Give Claude eyes with Playwright

Playwright MCP is Microsoft's official tool that lets Claude drive a real web browser: open pages, click buttons, fill forms, shrink to phone width and take screenshots.

Type **/exit** if you're inside Claude, then paste this in the plain terminal:

```
claude mcp add --scope user playwright -- npx @playwright/mcp@latest
```

Start Claude, type **/mcp**, and check that playwright shows as connected. Then test it inside Claude:

```
Use Playwright to open https://example.com, tell me the page title, and
take a screenshot.
```

A Chrome window pops up and Claude drives it. That's the moment this clicks for most people.

WINDOWS: IF PLAYWRIGHT SHOWS AS FAILED IN /MCP

Type /exit, run claude mcp remove playwright, then add it again with this version, which starts it through the Windows command runner: claude mcp add --scope user playwright -- cmd /c npx @playwright/mcp@latest

WHY THIS IS THE REAL UPGRADE

Without Playwright, Claude writes code blind and hopes it looks right. With it, Claude looks

at the page the way a visitor does, sees the overlapping text or the dead button, and fixes it before you ever see it. That one loop turns a demo into a site you can actually ship.

Get it right and stay safe

Skills and MCP servers are powerful because Claude follows them. That's also why you only install ones you trust. Here are the rules I use.

- **Install only from the exact sources in this guide.** Every repo here was checked: tens of thousands of stars, updated in the last few weeks, from the real author. Copycats with similar names exist.
- **Read Claude's permission prompts.** Claude Code asks before it runs a command or edits files. Read the one line it shows you. If it wants to touch something outside your website folder, say no and ask why.
- **Never let Playwright log into your real accounts.** It's a testing tool for your own site. Don't hand it your bank, email or ad account passwords.
- **Only authorize Figma on figma.com.** The Allow Access page must be on Figma's own website. Nothing else should ever ask for your Figma login.
- **Save checkpoints.** Git is a save-game system for code. Ask Claude to set it up before you start, so any bad change can be undone in one command.

```
Set up git in this folder and make a commit after every major step, with
a clear message, so I can roll back any change.
```

The real payload: the prompts that build the site

All of these go **inside Claude**, started from your my-website folder (cd my-website, then claude). Fill in everything in [square brackets] first. The more specific you are about your business, the less generic the result.

Prompt 1: The brief (sets the direction)

```
Use the design-taste-frontend skill. Before writing any code, read this
brief and give me 3 distinct design directions for the site. For each: a
name, the mood in one sentence, font pairing, color palette with hex
codes, layout style, and motion level. Then recommend one and say why.

Business: [what you do, in one sentence]
Who it is for: [your ideal customer]
What they should do on the site: [book a call / buy / sign up]
Brands whose look I like: [2-3 websites]
```

```
Vibe I want to avoid: [e.g. corporate, cheap, cluttered]


When I pick a direction, save it (fonts, colors, layout, motion, and the
Taste dials) to a file called DIRECTION.md so we can use it in later
sessions.
```

Not sure how to fill it in? Here's a real example of a good brief:

```
Business: family dental clinic in Austin, Texas
Who it is for: parents aged 30-45 who are nervous about dentists
What they should do on the site: book a first visit
Brands whose look I like: aesop.com, onemedical.com

Vibe I want to avoid: cold clinical blue-and-white stock photos
```

You'll get: three properly different looks with real fonts and hex colors. For that clinic, something like "Warm Neighborhood: soft cream and terracotta, a friendly rounded serif, big real-family photos, gentle motion." Pick one, or mix: "Direction 2, but with the palette from 1."

Why it matters: choosing the direction before building is exactly what an agency charges for in its discovery phase. You stay the decision maker.

Prompt 2: Build the full website

```
Read DIRECTION.md and PRODUCT.md, then build the full website in that
direction. Use the design-taste-frontend and impeccable skills for every
decision, and emil-design-eng for all motion.

Put all website files in a subfolder called site.
Pages: Home, About, Services, [Case Studies / Pricing], Contact.
Home sections: hero with one clear promise and one main button, social
proof, the problem we solve, how it works in 3 steps, services,
testimonials, FAQ, final call to action.
Tech: plain HTML, CSS and JavaScript, no build step, so the site folder
can be uploaded as-is. Fully responsive, from 375px phone to 1440px
desktop.
Contact form: make it work on Netlify Forms (add the data-netlify="true"
attribute and a hidden form-name field) and add a thank-you page.
Copy: write real, specific copy for my business, no lorem ipsum, no
buzzwords.
Images: use clearly labeled placeholder boxes with the exact size I
should replace them with.
```

```
Motion: subtle and fast. Ease-out on anything entering, nothing over
300ms, respect reduced-motion settings.

When done, start a local preview and give me the link to open in Chrome.
```

Why it helps: it names the skills, so Claude actually uses them, and it defines the pages, sections and tech up front so nothing gets skipped. Plain HTML means you can host it anywhere for free.

You'll get: a complete multi-page site in the site folder, plus a link like http://localhost:8000. Localhost just means "this computer", so only you can see it until you publish. The preview stops when you close Claude; just ask "start the preview again" next time.

Prompt 3: Tell Claude what you don't like

Click through the preview yourself first and write down everything that bugs you, in normal words. Then paste:

```
Here is what I don't like, in plain words:
[1. the hero feels empty]
[2. the green is too bright]
[3. the services section is boring]

Fix each one. Use Playwright to show me a before and after screenshot of
each fix, and don't change anything I didn't mention.
```

Why it helps: you don't need design vocabulary. "Feels empty" is enough, because the skills translate your gut reaction into concrete design fixes.

Prompt 4: The designer review (Impeccable)

The first build is a draft. These commands are the agency's review round, each pass done by a different specialist. Run them inside Claude, one at a time:

```
/impeccable critique home
/impeccable audit
/impeccable layout home
/impeccable typeset

/impeccable polish
```

You'll get: critique scores the design like a senior reviewer. Audit checks speed, phone layout and accessibility (whether people with poor eyesight or screen readers can use it). Layout fixes spacing, typeset

fixes fonts, and polish is the final pre-launch pass.

Prompt 5: The motion pass (Emil's skills)

```
Use the improve-animations skill to audit every animation on the site and
give me a prioritized list of fixes. Then apply them. After that, use
review-animations to check the result strictly against Emil's rules and
fix anything that fails.
```

Why it helps: motion is the difference between a site that feels cheap and one that feels expensive, and it's where AI is worst by default. This pass fixes the speed and feel of every movement (designers call that easing), the details visitors notice without knowing why.

Prompt 6: The Playwright test loop

```
Start the local preview, then use Playwright to test the whole site like
a picky first-time visitor. Repeat this loop until everything passes:

1. Open every page at 1440px wide, then at 375px wide (phone).
2. Take a screenshot of each page at both sizes and look at it carefully.
3. Click every link and button. Fill in the contact form with test data
and check it reaches the thank-you page.
4. Check the browser console for hidden error messages.
5. List every problem you find: broken links, overlapping or cut-off
text, sideways scrolling on phone, buttons too small to tap, missing
hover states, errors, slow or janky animation.
6. Fix all of them, then run the full check again.

Also use the break-ui skill to fill the pages with worst-case content
(very long names, empty lists, huge numbers) and fix anything that
breaks.

Stop only when a full pass finds zero problems. Then give me a short
report of what you found and fixed.
```

Why it helps: this is the QA step agencies bill hours for. The "stop only at zero problems" line is what makes Claude keep going instead of declaring victory early.

You'll get: a site that works on phone and desktop, and a plain report of every bug it caught and fixed.

Prompt 7: Get found on Google

```
Add SEO basics to the site: a unique title and meta description for every
page, a share image for when the link is posted on social media,
sitemap.xml, robots.txt, LocalBusiness schema with my business name,
address, phone and opening hours [fill these in], and descriptive alt
text on every image.
```

Why it helps: agencies include this on every site and most owners don't know it exists. It's how Google understands who you are and where you are.

Prompt 8: Swap in your real photos

Make a folder called photos inside my-website and drop your images in. Then paste:

```
My photos are in the photos folder. Replace each placeholder with the
best match, crop it to the labeled size, convert it to WebP under 200KB,
and write alt text for each one.
```

Prompt 9 (optional): The Figma round trip

Have a design in Figma? Right-click the frame, choose Copy link to selection, and paste it into this prompt:

```
Build this section to match the Figma design exactly, using our existing
styles: [paste Figma frame link]
```

Want the finished site as an editable Figma file for a designer or client? Use this:

```
Capture the home page of my running site into a new Figma file as
editable design layers.
```

Autopilot: the whole agency in one prompt

This is the "describe the site, Claude ships it" version from the reel. Once you've done one build by hand and you trust the setup, paste this inside Claude with your brief at the bottom:

```
Run the full website pipeline without stopping, using my brief below:
1. Use design-taste-frontend to pick the strongest direction and save it
to DIRECTION.md.
2. Build the full site in a subfolder called site (plain HTML, CSS and
JS, responsive, Netlify Forms contact form, thank-you page).
3. Run /impeccable critique, layout, typeset and audit, and fix what they
find.
4. Use improve-animations, then review-animations, and fix what they
find.
5. Run the Playwright test loop at 1440px and 375px until a full pass
finds zero problems, including break-ui worst-case content.
6. Add SEO basics: titles, descriptions, sitemap.xml, robots.txt,
LocalBusiness schema, alt text.
7. Run /impeccable polish.
Commit to git after every stage. Only stop to ask me if a decision
changes the brand.

Brief:

[paste your brief from Prompt 1]
```

Why the step-by-step prompts still exist: autopilot is fast, but you give up control. Use the separate prompts for your first site and for any client work, so you make the big calls yourself.

Launch: put it online for free

1. **Go to app.netlify.com/drop**
   Create a free Netlify account.
2. **Drag in the site folder**
   Open my-website, then drag only the site folder (not the whole project) onto the page. In a few seconds you get a live link ending in netlify.app.
3. **Switch on your contact form**
   In your new Netlify site, open Forms and click Enable form detection. Then drag the site folder in again (on the Deploys page), because detection only starts on the next upload.
4. **Test it live**
   Fill in your own form on the live link. The message shows up under Forms in Netlify, and you can turn on email notifications there.
5. **Update any time**
   Changed something? Drag the site folder onto the Deploys page again and it's live.

Want your own domain name? Ask Claude:

```
Walk me step by step through connecting my domain [yourdomain.com] to my
```

```
Netlify site.
```

Cheat sheet: Impeccable's 24 commands in plain English

Type each one as **/impeccable** followed by the command and, optionally, the page name, for example /impeccable bolder home. (The reel said 23. Impeccable has added more since, so you get 24.)

The 5 you'll actually use: critique, audit, layout, typeset and polish. Learn those first. The rest are here when you need them.

- **Planning:** craft (plans and builds a page with you), init (one-time setup), shape (plans the layout before any code), document (writes down the design rules of an existing site), extract (turns repeated pieces into reusable building blocks).
- **Reviewing:** critique (a design review), audit (speed, phone and accessibility checks), polish (the final pre-launch pass), harden (tests what happens with very long names, missing info or errors).
- **Changing the feel:** bolder (braver), quieter (calmer), distill (strips it to the essentials), colorize (adds strategic color), delight (adds small moments of joy), overdrive (show-off effects).
- **Fixing details:** layout (spacing and rhythm), typeset (fonts and hierarchy), clarify (rewrites confusing button and page text), onboard (what first-time visitors see when a page is empty).
- **Motion and speed:** animate (adds purposeful motion), optimize (makes the site load faster), adapt (fits the design to other screen sizes).
- **Trying variations:** live (try design variations right in the browser), generate (makes variations of one element automatically).

SHORTCUT

> Turn the command you use most into its own shortcut: /impeccable pin audit creates a plain /audit command.

Emil's motion rules (so you can spot cheap animation yourself)

These come straight from Emil Kowalski's published animation tips. Once you know them, you'll notice them on every premium site you visit.

1. **Ease-out for anything entering or leaving.**
   It starts fast and settles gently, so the page feels instant. Ease-in (slow start) feels laggy.
2. **Keep it under 300 milliseconds.**
   Fast motion feels responsive. Slow motion feels like waiting.
3. **Buttons shrink slightly when pressed.**
   A scale of about 0.97 on press gives instant, physical feedback.

4. **Never grow things from zero.**
   Start pop-ups at around 0.9 to 0.95 of their size, not 0. From zero, things look like they appear out of nowhere.
5. **Animate from where the click happened.**
   A dropdown should grow out of its button, not from its own center.
6. **Don't animate what people use 100 times a day.**
   Repeated animation gets annoying. Save motion for moments that matter.
7. **A tiny blur hides rough transitions.**
   A 2px blur during a switch blends the two states. Use it as a last resort, not a default.

Pro moves most people miss

- **Feed it references, not adjectives.** "Modern and clean" means nothing. "Spacing like stripe.com, type like linear.app" gives Claude a target it can hit.
- **Screenshot competitors with Playwright.** Ask Claude to open three competitor sites, screenshot them, and tell you what they all do the same, so your site does something different.
- **Turn the Taste dials.** The Taste Skill has three 1 to 10 settings: design variance, motion intensity and visual density. Tell Claude "variance 8, motion 4, density 3" for a bold but calm site.
- **Run critique before polish.** Polish makes things neat. Critique tells you if the idea itself is wrong. Fix the idea first.
- **Keep your decisions in files.** DIRECTION.md and PRODUCT.md are Claude's memory between sessions. If you change your mind about a color or the audience, ask Claude to update those files too.
- **Do a monthly check-up.** Once a month, ask Claude: "Open my live site with Playwright, check every link and the contact form, and report anything broken."

THE AGENCY SECRET

> Agencies don't win on code. They win on decisions: the direction, the copy, what to leave out. Claude now builds, you decide, and that's the part worth $10,000.

Common mistakes (and how to avoid them)

THE #1 MISTAKE: TYPING IN THE WRONG PLACE

> Install commands (npx, claude mcp add, claude plugin install) go in the plain terminal. Slash commands (/mcp, /impeccable, /plugin) and prompts go inside Claude. If something says "command not found" or Claude just chats back about your command, you're in the other place.

1. **One-line briefs.**
   "Build me a website for my agency" gets you the generic template. Always fill in the full brief from Prompt 1.

2. **Installing lookalikes.**
   Searching a skill name and installing the first result. Copy the exact commands from this guide, with the full GitHub addresses.
3. **Skipping the test loop.**
   The site looks great in one screenshot, then the phone menu doesn't open. Always run Prompt 6 before you share the link.
4. **Burning your Figma calls.**
   On a free Figma seat you get 6 a month. Don't connect Figma "just to see". Use it when you have a real design to build or hand off.
5. **Too much animation.**
   Everything sliding and bouncing looks like 2012. Run the motion pass and trust its cuts. Bounce and elastic easing are on Impeccable's banned list for a reason.
6. **Forgetting the form.**
   A pretty site whose contact form goes nowhere loses you leads. Do the Enable form detection step and send yourself a test message.

Your first week

1. **Day 1: Setup (about 30 minutes).**
   Checklist, Claude Code, the three skills, Playwright. Run the example.com test so you see Claude drive a browser.
2. **Day 2: Direction.**
   Write your full brief and run Prompt 1. Pick a direction (it gets saved to DIRECTION.md). Sleep on it.
3. **Day 3: Build.**
   Run Prompt 2. Click through the preview yourself and write down everything you dislike.
4. **Day 4: Review and motion.**
   Run Prompt 3 with your notes, then the Impeccable commands (Prompt 4) and the motion pass (Prompt 5).
5. **Day 5: Test.**
   Run the Playwright loop (Prompt 6) until it reports zero problems. Open the preview on your own phone too if you can.
6. **Day 6: Real content.**
   Run Prompt 7 (Google) and Prompt 8 (photos), reread every sentence of copy, then /impeccable polish one last time.
7. **Day 7: Launch.**
   Follow the launch steps, connect your domain, test the live form, and send the link to five customers for honest feedback.

THEN DO IT AGAIN

Your second site takes an afternoon, not a week, and the Autopilot prompt makes it faster still. That's when this stops being a fun experiment and starts being a skill people will pay you for.

Resources and Links

- Taste Skill on GitHub (Leonxlnx/taste-skill)
- Taste Skill website
- Impeccable on GitHub (pbakaus/impeccable)
- Impeccable docs and commands
- Emil Kowalski's skills on GitHub (emilkowalski/skills)
- Emil Kowalski's skill page
- Emil Kowalski: 7 practical animation tips
- Figma MCP server: official setup guide
- Figma MCP server: full tools list
- Playwright MCP on GitHub (microsoft/playwright-mcp)
- Claude Code install guide
- Using skills in the Claude app
- The skills installer (vercel-labs/skills)
- Node.js download
- Git for Windows
- Netlify Drop (free hosting)
- Netlify Forms setup

Want more AI workflows like this?
Join our free community for the best AI tools, prompts, and shortcuts.
Join here: skool.com/viral-ads